

Practical - 26: Implementation of the use of abstract class and interface

101) The abstract Vegetable class has three subclasses named Potato, Brinjal and Tomato. Write a program that demonstrates how to establish this class hierarchy. Declare one instance variable of type String that indicates the color of a vegetable. Create and display instances of these objects. Override the toString() method of object to return a string with the name of vegetable and its color.
(A)

```
abstract class Vegetable {
    String color;

    Vegetable(String color) {
        this.color = color;
    }

    public abstract String toString();
}

class Potato extends Vegetable {
    Potato(String color) {
        super(color);
    }

    public String toString() {
        return "Potato Color: " + color;
    }
}

class Brinjal extends Vegetable {
    Brinjal(String color) {
        super(color);
    }

    public String toString() {
        return "Brinjal Color: " + color;
    }
}

class Tomato extends Vegetable {
    Tomato(String color) {
        super(color);
    }

    public String toString() {
        return "Tomato Color: " + color;
    }
}

public class p101 {
    public static void main(String[] args) {
        Vegetable v1 = new Potato("Brown");
        Vegetable v2 = new Brinjal("Purple");
        Vegetable v3 = new Tomato("Red");
    }
}
```

	<pre> System.out.println(v1); System.out.println(v2); System.out.println(v3); } } </pre>
102)	Create interface <code>EventListener</code> with <code>performEvent()</code> method. Create <code>MouseListener</code> interface which inherits <code>EventListener</code> along with <code>mouseClicked()</code>, <code>mousePressed()</code>, <code>mouseReleased()</code>, <code>mouseMoved()</code>, <code>mouseDragged()</code> methods. Also create <code>KeyListener</code> interface which inherits <code>EventListener</code> along with <code>keyPressed()</code>, <code>keyReleased()</code> methods. WAP to create <code>EventDemo</code> class which implements <code>MouseListener</code> and <code>KeyListener</code> and demonstrate all the methods of the interfaces. (A)
	<pre> interface EventListener { void performEvent(); } interface MouseListener extends EventListener { void mouseClicked(); void mousePressed(); void mouseReleased(); void mouseMoved(); void mouseDragged(); } interface KeyListener extends EventListener { void keyPressed(); void keyReleased(); } class p102 implements MouseListener, KeyListener { @Override public void performEvent() { System.out.println("Event Performed"); } @Override public void mouseClicked() { System.out.println("Mouse Clicked"); } @Override public void mousePressed() { System.out.println("Mouse Pressed"); } @Override public void mouseReleased() { System.out.println("Mouse Released"); } } </pre>



```
}

@Override
public void mouseMoved() {
    System.out.println("Mouse Moved");
}

@Override
public void mouseDragged() {
    System.out.println("Mouse Dragged");
}

@Override
public void keyPressed() {
    System.out.println("Key Pressed");
}

@Override
public void keyReleased() {
    System.out.println("Key Released");
}

public static void main(String[] args) {
    p102 e = new p102();

    e.performEvent();
    e.mouseClicked();
    e.mousePressed();
    e.mouseReleased();
    e.mouseMoved();
    e.mouseDragged();
    e.keyPressed();
    e.keyReleased();
}
}
```

103) Implement the structure of multiple inheritance using class and interface in java. (B)

```
class Parent {
    void show() {
        System.out.println("Parent Class Method");
    }
}

interface Test {
    void display();
}

class Child extends Parent implements Test {
```

```
public void display() {
    System.out.println("Interface Method");
}

public class p103 {
    public static void main(String[] args) {
        Child c = new Child();

        c.show();
        c.display();
    }
}
```

- 104) The Transport interface declares a deliver () method. The abstract class Animal is the super class of the Tiger, Camel, Deer and Donkey classes. The Transport interface is implemented by the Camel and Donkey classes. Write a test program that initialize an array of four Animal objects. If the object implements the Transport interface, the deliver () method is invoked. (B)**

```
interface Transport {
    void deliver();
}

class Camel implements Transport {
    public void deliver() {
        System.out.println("Camel delivers goods");
    }
}

class Donkey implements Transport {
    public void deliver() {
        System.out.println("Donkey delivers goods");
    }
}

public class p104 {
    public static void main(String[] args) {

        Camel c = new Camel();
        Donkey d = new Donkey();

        c.deliver();
        d.deliver();
    }
}
```

- 105) Implement the structure of hybrid inheritance using class and interface in java. (C)**

```
class A {
    void showA() {
```

```

        System.out.println("Class A");
    }
}

interface B {
    void showB();
}

class C extends A implements B {
    @Override
    public void showB() {
        System.out.println("Interface B");
    }
}

public class p105 {
    public static void main(String[] args) {
        C obj = new C();

        obj.showA();
        obj.showB();
    }
}

```

106)

Declare a class called book having author_name as private data member. Extend book class to have two sub classes called book_publication & paper_publication. Each of these classes have private member called title. Write a program to show usage of dynamic method dispatch (dynamic polymorphism) to display book or paper publications of given author. Use command line arguments for inputting data. (C)

```

class Book {

    private String authorName;

    Book(String authorName) {
        this.authorName = authorName;
    }

    void display() {
        System.out.println("Author : " + authorName);
    }
}

class BookPublication extends Book {

    private String title;

    BookPublication(String authorName, String title) {
        super(authorName);
        this.title = title;
    }
}

```



```
void display() {
    System.out.println("Book Publication");
    System.out.println("Title : " + title);
}

}

class PaperPublication extends Book {

    private String title;

    PaperPublication(String authorName, String title) {
        super(authorName);
        this.title = title;
    }

    void display() {
        System.out.println("Paper Publication");
        System.out.println("Title : " + title);
    }
}

public class p106 {

    public static void main(String[] args) {

        if (args.length < 3) {
            System.out.println("Usage:");
            System.out.println("java p106 book AuthorName Title");
            return;
        }

        Book b;

        if (args[0].equalsIgnoreCase("book")) {
            b = new BookPublication(args[1], args[2]);
        } else {
            b = new PaperPublication(args[1], args[2]);
        }

        b.display();
    }
}
```